

Lab 3 - Scheduling with MPI

Large Scale Hybrid Computing

University of Minho

The Lab 3 focus on the development of scheduling strategies to distribute the execution of a set of dependent tasks. Use a Deucalion node with ARM CPUs (by specifying the partition `normal-arm` when submitting a job). Compile the code either in the frontend or on the compute nodes (the latter is advised).

This lab tutorial includes five exercises to be solved during the lab class. Copy the source files, as usual, from `/projects/F202500010HPCVLABUMINHO/LSHC/lab3` to your home directory in Deucalion:

```
cp -r /projects/F202500010HPCVLABUMINHO/LSHC/lab3 .
```

To load the compiler in the environment use the following commands (do not forget to also use them in your submission script):

GNU Compiler: `module load GCC/14.3.0.`

MPI Libraries: `module load OpenMPI/5.0.8-GCC-14.3.0.`

You can use other GNU/OpenMPI combinations at your own risk, but ensure that OpenMPI is compiled with the correct version of the compilation toolchain.

1. Analyse and setup the codebase

Goals: to understand the creation and manipulation of DAGs.

Lab 1.1 Consider and analyse the provided codebase. It contains several functions to be completed throughout this lab guide. The code uses a set of data structures to manage task execution, while respecting their dependencies, from which the most relevant are `dag` and `indegree`. The former holds the dependencies among tasks, while the latter holds the amount of dependencies that still must be executed before a given task.

Start by executing the code as it is and measure the execution time. Complete the code required for the `calculateMakespan` function and compare its output with the measured execution time. Are there any differences? If so, is the source heterogeneity in the hardware or the irregularity of the tasks?

Lab 1.2 Understanding the chain of dependencies among tasks is a requirement for advanced scheduling strategies. Complete the `printAllDependencyChains` function (if you're unsure of its purpose ask an LLM or something), where the chains can be calculated by calculating all possible paths in the dependencies DAG. Consider using a Depth-First Search or a Breadth-First Search to traverse the graph.

Lab 1.3 A set of tasks that is applied to a independent dataset can be scheduled in two different ways:

- (i) A task is not only a the function itself, but a combination of the function and its dataset element. The n tasks to schedule would not only be $T_1, T_2, \dots, T_n$, but $T_{1_1}, \dots, T_{n_1}, \dots, T_{1_i}, \dots, T_{n_i}$, where i is the current dataset element.
- (ii) A pipeline of tasks is considered for each dataset element, where the whole pipeline is scheduled at once to a worker and is indivisible.

Identify the pros and cons of each approach considering regular and irregular workloads.

To implement (ii) it is required to calculate a path that traverses through all tasks (even the ones without dependencies) while minimising the pipeline execution time. Create (or adapt) a new DAG structure that allows weights to be added in the edges between nodes. Dependencies between tasks can be coded as an infinite weight in the connecting directed edge. Create a function that traverses the new DAG and outputs an optimal pipeline order.

2. Schedule the tasks

Goals: to develop skills task scheduling.

Lab 2.1 The provided skeleton includes a round-robin scheduler, which requires the worker to interact at each processed task. Analyse the code and create a new scheduler work-sharing scheduler, where the master process gives chunks of tasks to the workers. Once the worker is done with its set of tasks it should ask the master for more, or finalise. Find the easiest strategy to respect task dependencies (ask for tips)! It may require modifications to the worker's code.

Lab 2.2 Consider exercise 1.3. Develop a scheduler that computes the ideal pipeline and schedules its execution among workers, statically and in pre-defined chunks of elements. Consider that the pipeline processes 1k imaginary dataset elements (i.e., ensures it executes 1k times).